

Ahoy: LLMs Enacting Multiagent Interaction Protocols

Omkar Joshi¹[0009–0006–5069–2326], Amit K. Chopra²[0000–0003–4629–7594], and
Munindar P. Singh¹[0000–0003–3599–3893]

¹ North Carolina State University, Raleigh, NC, USA

² Lancaster University, Lancaster, UK

MPS 1: * refers to my advice pages

MPS 2: G refers to grammar

Abstract. Multiagent systems typically require protocol-specific agent implementations: a change in the protocol necessitates different agents or extensive code modifications. This tight coupling makes it difficult to build flexible agents. We demonstrate that *Large Language Models* can overcome this limitation by enabling a single agent to correctly enact multiple protocols without protocol-specific engineering. We introduce *Ahoy*, an approach that enables LLM-driven agents to select and enact multiple interaction protocols guided by natural language input from users. Ahoy creates agents that are protocol-agnostic — operating without protocol-specific code — while respecting BSPL protocol constraints that ensure correct behavior. Our key contribution is a generic agent adapter (Ahoy) that decouples agent decision making from protocol specification. We evaluate Ahoy across two distinct protocols—Logistics (packaging and shipping coordination) and Purchase (buyer-seller-shipper coordination).

Our results show that Ahoy-enabled LLMs correctly select and enact protocols without specialized training, producing no adapter exceptions, malformed messages, or constraint violations. Ahoy also supports the construction of agents that can intelligently exploit flexible protocols. This approach reduces knowledge engineering overhead by enabling agents to participate in multiple protocols, demonstrating how LLM-enabled multiprotocol architectures enable adaptive coordination in evolving business domains, addressing a key challenge for scaling multiagent systems.

Keywords: Multiagent Systems

1 Introduction

Agentic AI is the idea that The promise of agentic AI is ...

Agentic AI is a multiagent enterprise. Agent interaction protocols are therefore crucial. Several protocols have been proposed to support agent interaction, e.g., A2A. They have limitations...

A major limitation of the traditional approach to MAS is that execution of new protocols require the development of new agents.

Recent advances in generative AI have led to the development of Large Language Models that demonstrate reasoning ability.

An LLM agent is ... We investigate the possibility of whether LLM agent may execute information protocols correctly and in a manner that aligns with its user's goals.

Research Question: *Can LLM agents effectively select and execute multiple protocols without protocol-specific training or fine-tuning?*

Our key contributions are as follows:

- A multiprotocol selection framework to detect appropriate protocol based on natural language input.
 - A generic agent implementation (Ahoy) that can participate in multiple protocols with minimal code changes.
 - Empirical Evaluation across two/three protocols
- OJ 1: [subject to time constraint](#)

2 Background

2.1 Multiagent Coordination — Specifying Interaction Protocols

Formally specified rules of encounter, e.g., in BSPL, define the messages, roles, and constraints that guide agent interactions. Traditional approaches require agent implementations to be fixed at design time and changes in the requirement or protocol require agent-level code changes.

BSPL (Blindly Simple Protocol Language) is a declarative specification language where protocols are described in terms of roles, messages, and logical constraints. OJ 2: [add bspl explanation](#)

OJ 3: [add bspl protocol example](#)

2.2 Programming Agents Based on Interaction Protocols

Kiko is a framework for implementing protocol-based agents. It provides an adapter that exposes a Python API based on decorators for implementing agents. The API allows developers to plug in the agent's reasoning (e.g. when to take an action or send messages), while the adapter handles protocol compliance.

OJ 4: [enhance kiko explanation](#)

OJ 5: [insert Kiko example here](#)

2.3 Programming LLM Agents

Large Language Models can be used as the reasoning component in agent architectures. Programming LLM agents typically involves:

- System Prompts
- User Prompts
- Tool Calling
- Memory (RAG/Agent Notes)

These components work together to create an agentic loop where the LLM observes state, reasons about options and takes actions.

[OJ 6: Enhance the programming components descriptions](#)

[OJ 7: Add a simple example of LLM programming here](#)

3 Architecture

Ahoy achieves protocol agnosticism by decoupling LLM reasoning from protocol-specific logic. Rather than implementing distinct agents for each protocol, Ahoy uses a unified architecture where the LLM serves as the decision-making layer, reasoning over constraints and possibilities defined by BSPL protocols. The architecture orchestrates interaction between four key components through a well-defined flow:

1. **Initialization Phase (One-Time):** When a user provides a natural language goal, the *role detection module* (part of the LLM decision layer) queries the LLM to select an appropriate protocol and role from available options. This selection is fixed for the lifetime of the agent instance. Once the protocol and role are determined, a Kiko adapter is instantiated for that specific protocol-role pair, and the prompt construction layer generates a system prompt containing foundational knowledge about BSPL semantics and the selected protocol.
2. **Execution Loop (Repeated):** During protocol enactment, the Kiko adapter monitors protocol state and triggers a decision event whenever state changes (e.g., a message arrives from another agent or a custom event fires). The decision endpoint (mediator between LLM and adapters) receives this event and invokes the prompt construction layer to assemble a context-aware user prompt from the current adapter state. The user prompt is passed to the *message decision module* (part of the LLM decision layer), which selects which enabled message to send and how to bind its parameters. The decision endpoint returns this choice to the adapter, which validates and sends the message.
3. **Termination Check (Each Iteration):** Before each new decision, the *termination condition detection module* (part of the LLM decision layer) checks whether the protocol should conclude. Termination can result from

either protocol-driven completion (adapter detects terminal state) or user-defined criteria (e.g., threshold met, timeout reached). If termination conditions are satisfied, the protocol enactment concludes and the adapter shuts down gracefully.

4. **Concurrency (Optional):** Multiple Kiko adapters can execute concurrently, each managing isolated social state for a different protocol instance. The decision endpoint can interleave decisions across adapters, with the LLM receiving distinct prompts for each protocol context.

This design separates concerns across four layers: the LLM decision layer handles all reasoning (protocol selection, termination detection, message choice), the prompt construction layer transforms protocol state into LLM input, the decision endpoint orchestrates communication flow, and the Kiko adapters enforce all structural constraints and manage protocol state. This separation enables protocol agnosticism: new protocols require only a new BSPL specification file, not new LLM logic. The architecture is illustrated in Figure 1. The remainder of this section details each component’s role in this flow.

3.1 LLM Decision Layer

The LLM decision layer is responsible for all reasoning about protocol selection and execution. It consists of three interdependent modules that operate in sequence during agent initialization and execution.

Role Detection Module The role detection module determines which protocol and role combination is appropriate for a given user intent. When an agent is initialized with a natural language goal (e.g., “I want to buy a notebook”), this module queries the LLM with the set of available protocols, their roles, and the user goal. The LLM selects the protocol-role pair that best aligns with the stated objective. This determination happens once during initialization and is then fixed for the lifetime of that agent instance, ensuring consistent role behavior throughout execution.

Termination Condition Detection Module The termination condition detection module monitors whether a protocol enactment should conclude. Termination can occur for two reasons: (1) the protocol reaches a terminal state (successful completion or deadlock detected by the Kiko adapter), or (2) user-defined termination criteria are met (e.g., a transaction value exceeds a threshold, a timeout is reached, or a satisfaction condition is met). This module queries the current protocol state, consults any external conditions or alerts, and signals the adapter to halt gracefully when appropriate. By separating termination detection from the message decision logic, Ahoy can handle both protocol-driven and domain-driven conclusion criteria.

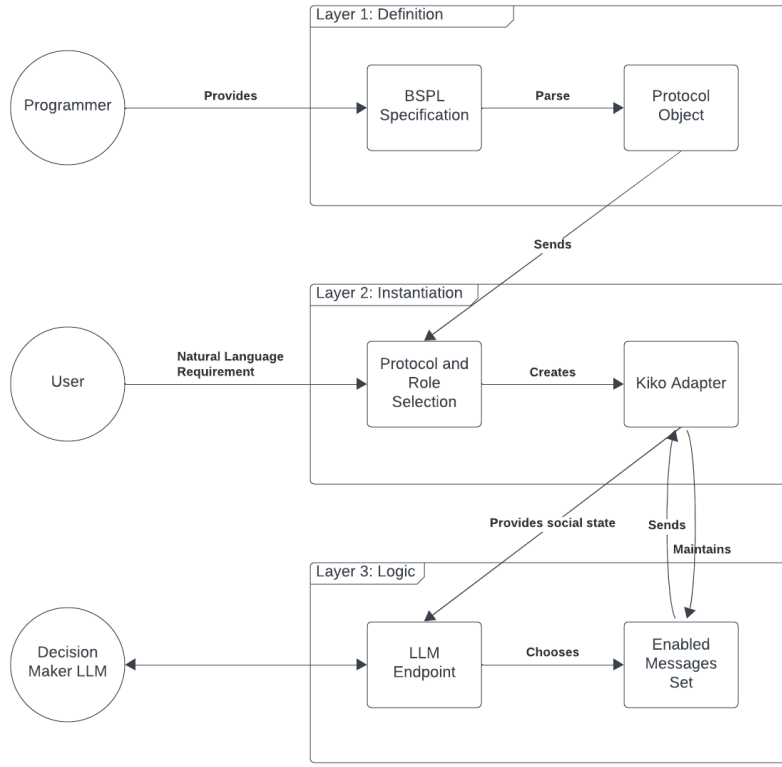


Fig. 1. Ahoy Architecture Overview

Message Decision and Parameter Binding Module The message decision module is the core reasoning component that executes repeatedly during protocol enactment. When a decision event is triggered (either by an incoming message or a custom event), this module receives: (1) the set of enabled messages from the Kiko adapter (all messages that can legally be sent without violating protocol constraints), (2) the current message history and bound parameters, and (3) agent goals and domain-specific preferences. The LLM evaluates the enabled options and selects the message(s) to send along with appropriate parameter bindings. The module is constrained to choose only from the enabled message set; the Kiko adapter ensures the preconditions are met, so the LLM need only reason about domain appropriateness and goal alignment.

3.2 Prompt Construction Layer

The prompt construction layer transforms protocol state into structured, context-aware LLM input. Prompts are assembled in two phases: a static system prompt

constructed once per protocol enactment, and dynamic user prompts constructed on each decision event.

System Prompt The system prompt, created once when the protocol and role are determined, establishes foundational knowledge that persists across all decision iterations. It contains: (1) an explanation of BSPL semantics (roles, messages, parameter binding rules), (2) the agent’s assigned role and protocol, (3) an explanation of enabled messages and how constraints work, (4) available tools and their calling conventions, and (5) human-readable guidelines for parameter binding. The system prompt is cached and reused to minimize API calls and maintain consistency. It is overwritten only when a new protocol enactment begins.

User Prompt For each decision event, a fresh user prompt is constructed by querying the Kiko adapter for current state. The user prompt is assembled from: (1) a role reminder establishing the agent identity and decision task, (2) the complete message history from the adapter showing all messages sent and received, (3) currently enabled messages extracted from the adapter’s enabled message store, (4) bound parameters already set by prior messages, and (5) which parameters must be filled for each enabled message option. This prompt encodes sufficient context for the LLM to make informed decisions while remaining concise enough to stay within token limits.

3.3 Decision Endpoint

The decision endpoint is an event handler that mediates between the Kiko adapter and the LLM decision layer. It is registered as a callback with the adapter and triggered whenever protocol state changes. The decision endpoint executes the following sequence: (1) receive a trigger event (message received, custom timeout fired, etc.) from the adapter, (2) invoke the termination detection module to check if the protocol should conclude, (3) if not terminating, construct the user prompt from current adapter state, (4) call the LLM decision module with the assembled system and user prompts, (5) parse the LLM’s response to extract selected message type and parameter bindings, (6) return the selected message instance to the adapter, which sends it according to protocol constraints. Because the adapter enforces all structural constraints, the decision endpoint can focus purely on domain reasoning without duplicating validation logic. This separation of concerns is the key to protocol agnosticism: the same endpoint logic works for any protocol.

3.4 Kiko Adapters (n Instances)

Ahoy instantiates one Kiko adapter per protocol-role pair being executed. Each adapter is independent and maintains isolated social state, ensuring that concurrent protocol enactments do not interfere.

Adapter Responsibilities Each Kiko adapter is responsible for: (1) enforcing protocol constraints (message schema validation, precondition checking, state machine transitions), (2) tracking message history and parameter bindings in isolated social state, (3) computing the set of enabled messages at each step (all messages whose preconditions are satisfied), (4) sending and receiving messages according to protocol rules, (5) detecting protocol terminal states (completion or deadlock), and (6) invoking registered decision endpoint callbacks when state changes. The adapter provides a Python API (via decorators such as `@adapter.reaction(MessageType)`) that allows agents to register handlers for protocol events.

Social State Isolation Because each protocol instance has its own adapter, parameters are naturally scoped per protocol. For example, an `orderId` parameter in the Purchase protocol is isolated from an `orderId` in the Logistics protocol; they do not interfere even though they share the same name. This isolation is enforced at the adapter level and requires no special logic in the LLM layer. Message history is similarly isolated: the LLM receives only the message history relevant to the current protocol when constructing user prompts.

Concurrent Execution Multiple adapters can run concurrently, enabling a single LLM-driven agent to participate in multiple protocol enactments simultaneously. Each adapter maintains its own decision endpoint callback and triggers decisions independently. The LLM receives distinct prompts for each protocol context and reasons appropriately for each. Concurrency is managed via asynchronous I/O (`asyncio`) to prevent blocking between protocol instances.

4 Programming Model

A key contribution of Ahoy is a programming model centered on the LLM decision endpoint pattern, which enables protocol-agnostic agent implementations. This programming model can be used at three distinct levels of expertise. At the *user* level, end users interact with pre-configured agents and protocols through a dialog interface without writing any code. At the *protocol* level, domain experts can define new protocols and configure agents for these protocols using declarative syntax while writing minimal code. At the *framework* level, we provide support for developers to extend Ahoy by implementing custom event handlers, decision endpoints, or new LLM tooling.

4.1 Level 1: End Users

End users interact with Ahoy agents through the dialog interface. No code knowledge is required. Users can describe their goals in natural language (e.g., "I want to buy a pen and deliver it to [Address]") and Ahoy uses the LLM-based protocol and role selection module to map the user intent to the appropriate protocol and role, then launches the agent automatically. This interface is provided by Ahoy: users do not need to understand protocols, adapters, or LLM endpoints.

4.2 Level 2: Domain Experts

Domain experts can define new protocols and configure agents for them using Ahoy. This level also requires no python coding — declarative BSPL syntax is sufficient. Protocol developers:

1. **Write Protocol Specifications:** Define roles, messages (with typed input and output parameters), and constraints in BSPL (see Section 2). For example, to add a new contract negotiation protocol, a domain expert writes the appropriate BSPL specification defining the Proposer and Acceptor roles, the messages they exchange, and the in/out parameters that govern message ordering.
2. **Register Protocol Metadata:** Add a one-line protocol description to the appropriate file that can be read by Ahoy and serves as an input to the protocol and role selection module.
3. **Configure Agent Behavior:** Specify agent goals and constraints in a human-readable input file (e.g, "Accept offers above \$100", "Propose at least a 10% markup"). These constraints are provided to the LLM as part of the system prompt.
4. **Test and Deploy:** Ahoy automatically handles protocol validation, adapter instantiation, and LLM integration. No code changes are required.

In this manner, Ahoy enables domain experts to focus solely on the protocol structure and agent goals, without the need to handle any implementation details.

4.3 Level 3: Framework Developers

Developers can extend Ahoy by adding new tools for LLM use, custom events that can trigger an LLM call, or custom decision strategies that do not need LLM integration. In order to modify Ahoy, developers need to understand the decision endpoint pattern shown in Listing 1.1.

Listing 1.1. Ahoy Decision Endpoint Pattern

```
from bspl.adapter import Adapter
from lib.state_manager import serialize_adapter_state
from lib.llm_client import AnthropicLLMClient

adapter = Adapter(Seller, systems, agents)
llm_client = AnthropicLLMClient()

async def llm_decision(enabled_store, event):
    # Serialize current protocol state
    state = serialize_adapter_state(adapter)

    # Get available messages from enabled_store
    enabled_messages = list(enabled_store.messages())
```



```

# Delegate reasoning to LLM
llm_response = await llm_client.complete(
    messages=[{
        "role": "user",
        "content": f"State:_{state}._"
                    f"Enabled_messages:_{enabled_messages}._"
                    f"Goal:_{Choose}_{action}_{price}_{10-100}._"
                    f"What_message_to_send?"
    }]
)

# Parse LLM response and bind parameters
decision = parse_llm_response(llm_response)
message_instance = bind_message(decision,
                                enabled_store)

return message_instance

# Register handler with decision endpoint
adapter.decision()(llm_decision)

```

5 Demos

Our experiments demonstrate three key capabilities of Ahoy:

1. Protocol Portability: The same agent code executes correctly across different protocols without modification.
2. Enforcement of Guarantees: Structural guarantees are maintained in all enactments.
3. Protocol Selection Accuracy: The protocol selection module reliably maps user intent to appropriate protocols and roles.

5.1 Demo 1: Protocol Portability

Goal: Demonstrate that a single Ahoy agent can enact multiple protocols without code changes.

Method: Execute identical agent decision logic in two distinct protocol contexts:

- Purchase Protocol (Buyer role): Agent participates in a buyer–seller–shipper negotiation.
- Logistics Protocol (Merchant role): Agent coordinates packaging and shipping workflow.

Both enactments use the same underlying LLM decision endpoint pattern shown in Listing 1.1. No protocol-specific logical branches or conditional code paths

are used.

Results: We capture execution traces from both protocols and verify the following.

- Both protocols reach terminal state.
- Message sequences respect protocol-specific constraints.
- No adapter exceptions or schema violations occur.
- LLM decision quality is consistent across both domains. **OJ 8: no stupid decisions**

5.2 Demo 2: Guarantee Validation

Goal: Verify that the framework guarantees hold across protocols.

Method: We execute curated scenarios targeting the core guarantees:

- **Message Validity:** Verify that only schema-conforming messages are offered to the LLM.
- **Parameter Scoping:** Test scenarios where identical parameter names (e.g. orderID) appear in two or more protocols; verify that they remain isolated and do not contaminate across protocol contexts.
- **Role Consistency:** Confirm that only role-appropriate messages appear in the enabled message set at each decision point.

Results: We produce a trace validation report documenting:

- Scenario description
- What would break in a naive agent
- How Ahoy handles it.
- Exceptions noted.

5.3 Demo 3: Concurrent Multiprotocol Participation

Goal: Demonstrate that a single LLM-driven agent can simultaneously participate in multiple protocol instances while maintaining state isolation and making coherent decisions across different protocol contexts.

Method: We instantiate parallel protocol enactments where the agent operates in distinct roles across multiple protocols concurrently:

- **Scenario Setup:** Agent acts as Buyer in Purchase protocol while simultaneously acting as Merchant in Logistics protocol.
- **State Isolation:** Each protocol maintains an independent adapter with isolated social state (message history, parameter bindings, enabled messages).
- **Decision Interleaving:** We trigger decision events across both protocols in an interleaved fashion, simulating realistic concurrent coordination.
- **Context Switching:** The LLM receives distinct user prompts for each protocol, including protocol-specific enabled messages and agent goals, testing whether the LLM maintains context separation.

Results: We capture execution traces from both concurrent protocol enactments and verify:

- **Parameter Isolation:** Parameters bound in one protocol (e.g., orderID in Purchase) do not interfere with identically-named parameters in another protocol (e.g., orderID in Logistics).
- **Message History Separation:** Each protocol’s message history remains isolated; the LLM references only relevant messages when making decisions in each context.
- **Coherent Decision Making:** Decisions made in one protocol do not violate constraints or create contradictions in the other protocol.
- **No Cross-Protocol Contamination:** Enabled message sets correctly reflect only messages available in each specific protocol context.
- **Concurrent Termination:** Both protocols reach terminal state independently without deadlock or state corruption.

5.4 Demo 4: Decision Quality Across Domains

Goal: Show that the LLM reasoning adapts seamlessly and makes semantically appropriate choices in different protocol contexts.

Method: We capture decision point snapshots from representative protocol enactments:

- Purchase Protocol: LLM chooses the appropriate price to buy items.
- Logistics Protocol: LLM chooses the appropriate material to wrap the package based on frailty without explicit guidance.

Results: Table showing

- Scenario description
- LLM action and reasoning
- Semantic appropriateness of decision in context (LLM as judge?)

5.5 Demo 5: Protocol Selection Accuracy

Goal: Validate that the protocol selection module reliably maps user intent to correct protocol–role pairs.

Method: We construct a test set of n user intent statements, including:

- Clear Purchase Intents: I want to buy a notebook
- Clear Logistics Intents: I want to organize packaging and shipping for a warehouse containing the following items ...
- Ambiguous intent: I want to ship something

Each test case has a ground truth protocol label.

Results: A results table showing

- User input
- Ground truth label
- Difficulty marker
- Selection output

and a summary table showing the accuracy, recall and precision.

5.6 Demo 6: Custom LLM Events

Goal: Demonstrate that Ahoy agents can respond to both protocol-driven events (received messages) and custom business logic events (timeouts, thresholds, periodic checks) using the same LLM decision endpoint.

Method: We instantiate agents that handle concurrent event sources:

- **Adapter Reactions:** Traditional protocol event handlers triggered when messages arrive.
- **Custom Events:** Business logic triggers such as periodic timeout checks, stall detection, or out-of-stock alerts.
- **Synchronization:** Both event paths converge on the same `choose_and_bind()` decision logic, synchronized via `asyncio.Lock` to prevent concurrent access conflicts.

Test scenarios include:

- **Purchase Protocol (Buyer with periodic timeout):** Adapter processes incoming quotes; custom event handler checks every 2 seconds whether to send follow-up messages.
- **Logistics Protocol (Merchant with stall detection):** Adapter processes shipping requests; custom event handler detects prolonged inactivity and triggers proactive decisions.

Results: We verify that:

- **Concurrent Execution:** Both adapter and custom event paths execute without race conditions or deadlocks.
- **Unified Decision Logic:** Both event types use identical parameter-binding and message-sending workflows.
- **Constraint Preservation:** Messages from both paths respect protocol constraints and parameter bindings.
- **Metric Tracking:** LLM call counts correctly reflect decisions from both adapter and custom event handlers.
- **Graceful Termination:** Agents exit cleanly when protocols complete or thresholds are exceeded.

5.7 Summary

Together, these experiments demonstrate that Ahoy achieves protocol-agnosticism. A single, generic agent implementation using Ahoy, correctly selects and enacts multiple protocols, fulfills all structural guarantees, and makes semantically sound decisions without protocol-specific code modifications.

6 Related Work

This section positions our work within the broader landscape of multiagent systems, LLMs and agent programming models.

6.1 LLMs in Multiagent Systems

Recent work explores the use of Large Language Models as reasoning engines in multiagent systems. Key approaches for LLM interaction include:

- In Context Learning
- Fine Tuning
- Chain of Thought Reasoning

In addition, there has been a push towards improving communication between LLM agents. Various approaches define protocols and communication patterns that enable LLM-based agents to coordinate effectively. Our work differs from existing approaches by focusing on *dynamic protocol selection and enactment*: given user intent expressed in natural language dialog, our framework automatically selects from multiple available protocols and binds the agent to the appropriate role without requiring protocol-specific agent implementations. **OJ 9: Explain how we differ from each prompting approach**

6.2 Programming Models

Established programming models for multiagent systems include the Actor model, reactive agents and BDI (Belief–Desire–Intention) architectures. These provide different abstractions for agent behavior and coordination. Our Ahoy framework complements these by ...

[?]